Internal Security Audit Report: RWA T-Bill Tokenization Platform

Audit type: Internal team-authored security review
Course: Blockchain Technologies 2, Final Project
Scenario: Option C - RWA Tokenization Platform
Review target: Foundry contracts, deployment scripts, tests, subgraph events, and frontend transaction surfaces
Network target: Base Sepolia, with Anvil local deployment for reproducible demos
Auditors: Tokhtamishov Bekhruz and Zhangir Muslimov
Status: Ready for instructor review after final Base Sepolia deployment secrets are provided.

1. Executive Summary

This report covers the RWA T-Bill Tokenization Protocol. The protocol includes an upgradeable asset-backed ERC20, governance token, ERC721 reserve certificates, ERC4626 vault, from-scratch AMM, Chainlink-compatible oracle adapter, pull-payment treasury, factory, deployment scripts, subgraph, and frontend.

No Critical, High, or Medium issues are currently known from the internal review. The test suite includes two reproduced-and-fixed vulnerability case studies: reentrancy and missing access control. The repository is configured so CI runs Slither and fails submission quality checks if High or Medium findings are introduced.

0
Known Critical, High, or Medium internal findings at the time of this report.

2
Before/after vulnerability case studies reproduced in Foundry tests.

88
Automated tests across unit, fuzz, invariant, and fork-hook coverage.

Overall Risk Rating

Category	Rating	Rationale
Smart-contract correctness	Low	Core flows are covered by unit, fuzz, invariant, and fork tests.
Governance centralization	Medium	Timelock controls privileged roles; delay reduces but does not remove plutocratic governance risk.
Oracle dependency	Medium	Chainlink-compatible feeds are validated for freshness, but external feed correctness remains an assumption.
Testnet reserve proof	Low	Mock reserve feed is clearly documented for Base Sepolia demo and not represented as production collateral.
Frontend risk	Low	Frontend does not custody funds; it formats contract reads/writes and user-facing transaction errors.

2. Scope

Commit and Environment

Commit hash: to be filled at final submission after the last commit.

Review environment:

- Foundry project only. No Hardhat config, dependency, deployment script, or CI step.
- Local chain: Anvil chain ID 31337.
- Target testnet: Base Sepolia.
- Production dependencies: OpenZeppelin contracts and contracts-upgradeable 5.6.1.

Files In Scope

Area	Files
Asset token	`src/TBillToken.sol`, `src/TBillTokenV2.sol`
Governance token	`src/RwaGovernanceToken.sol`
Reserve evidence	`src/ReserveCertificateNFT.sol`
Vault	`src/TBillVault.sol`
AMM	`src/RwaStableAMM.sol`, `src/libraries/RwaMath.sol`
Oracle	`src/OracleAdapter.sol`, `src/interfaces/AggregatorV3Interface.sol`
Treasury	`src/ProtocolTreasury.sol`
Deployment infra	`src/ProtocolFactory.sol`, `script/Deploy.s.sol`, `script/UpgradeTBillToken.s.sol`, `script/VerifyDeployment.s.sol`
Governance	`src/RwaGovernor.sol`
Tests	`test/Protocol.t.sol`, `test/invariants/ProtocolInvariants.t.sol`, `test/fork/ForkIntegrations.t.sol`, fixtures.

Files Out of Scope

Area	Reason
OpenZeppelin libraries	Treated as trusted audited dependencies.
forge-std	Test-only framework dependency.
RPC providers	Infrastructure dependency; downtime or incorrect RPC responses are not controlled by the protocol.
Wallet providers	User wallet implementation and browser extension security are outside protocol control.
Legal reserve custody	Off-chain custody and legal ownership of T-Bills are outside smart-contract scope.
The Graph infrastructure	Subgraph indexing is read-only and does not affect on-chain state.

3. Methodology

The review combined manual inspection, automated tests, fuzzing, invariant testing, fork integration hooks, coverage measurement, and static-analysis configuration.

Manual Review Checklist

Review area	Procedure
Access control	Confirm every privileged external function is guarded by OpenZeppelin `AccessControl` or Governor.
Reentrancy and CEI	Inspect all value-moving flows for CEI ordering or `ReentrancyGuard`.
ERC20 safety	Confirm protocol ERC20 movements use `SafeERC20`.
Oracle safety	Confirm stale, zero, and negative answers revert.
Upgrade safety	Confirm implementation disables initializers and V2 appends storage only.
Governance safety	Verify voting delay, voting period, quorum, threshold, and Timelock delay.
Treasury accounting	Confirm owed balances are zeroed before external calls.
AMM invariants	Confirm swaps enforce slippage and do not decrease constant-product invariant after fee handling.
Forbidden primitives	Search for `tx.origin`, `transfer`, `send`, and timestamp randomness.

Automated Commands

```
forge build
forge test
forge coverage --ir-minimum --report summary --no-match-coverage 'script|test|src/mocks'
forge fmt --check
solhint 'src/**/*.sol' 'script/**/*.sol' 'test/**/*.sol'
slither . --filter-paths "test|script|node_modules|lib" --exclude-dependencies
npm run guard:no-hardhat
npm --prefix frontend run build
npm run subgraph:build
```

Slither is installed and run in GitHub Actions with `pipx install slither-analyzer`. The local machine used during implementation did not include a preinstalled `slither` binary, so the final Slither appendix should be filled with CI output from the final commit.

4. Findings Table

ID	Severity	Title	Location	Status
C-01	Critical	None identified	N/A	N/A
H-01	High	None identified	N/A	N/A
M-01	Medium	None identified	N/A	N/A
L-01	Low	Base Sepolia reserve feed is mock-backed	`script/Deploy.s.sol`	Acknowledged
L-02	Low	Emergency pause remains a powerful governance capability	`TBillToken`, `TBillVault`, `ReserveCertificateNFT`	Acknowledged
L-03	Low	Testnet deployment is not a legal proof of real-world custody	Deployment and README	Acknowledged
I-01	Informational	Coverage uses `--ir-minimum` to avoid deploy-script stack depth	`package.json`, CI	Acknowledged
I-02	Informational	Fork tests require `MAINNET_RPC_URL` to exercise live integrations	`test/fork/ForkIntegrations.t.sol`	Acknowledged
G-01	Gas	Yul sqrt helper is used in AMM initial liquidity	`src/libraries/RwaMath.sol`	Fixed / benchmarked

5. Finding Details

L-01: Base Sepolia Reserve Feed Is Mock-Backed

Severity: Low.

Location: `script/Deploy.s.sol`.

Description: The Base Sepolia deployment path supports Chainlink-compatible mock feeds for price and reserve data. This is acceptable for course and testnet demonstration, but it must not be presented as production proof of real T-Bill reserves.

Impact: Users could misunderstand a testnet reserve answer as real-world collateral if the limitation is not documented.

Proof of concept: The deploy script creates mock aggregator contracts when official testnet feeds are unavailable.

Recommendation: Before mainnet deployment, replace mock feeds with official Chainlink price/proof-of-reserve feeds or a production-grade attestation provider through Timelock governance.

Status: Acknowledged and documented.

L-02: Emergency Pause Is Governance-Controlled

Severity: Low.

Location: `TBillToken`, `TBillVault`, `ReserveCertificateNFT`.

Description: Pause roles can temporarily halt transfers, certificate flows, and vault operations.

Impact: A malicious governance majority or compromised governance process could pause user flows. This does not steal funds directly, but it affects availability.

Proof of concept: Contracts expose `pause` and `unpause` functions guarded by pauser roles.

Recommendation: Keep pause authority behind Timelock, monitor proposals, and document the operational runbook for emergency pause usage.

Status: Acknowledged.

L-03: Testnet Deployment Is Not Legal Custody Proof

Severity: Low.

Location: README, architecture document, deployment scripts.

Description: The protocol models RWA flows but cannot by itself prove legal ownership of off-chain T-Bills.

Impact: A deployed ERC20 can be technically correct while still lacking legally enforceable reserve backing.

Proof of concept: Smart contracts enforce roles and feed freshness, but legal title, custody, and reserve audits occur off-chain.

Recommendation: Treat reserve certificates as evidence references, not legal title. Add production legal agreements and verified reserve providers before mainnet launch.

Status: Acknowledged.

I-01: Coverage Uses `--ir-minimum`

Severity: Informational.

Location: `package.json`, `.github/workflows/ci.yml`.

Description: The coverage command uses `--ir-minimum` because coverage disables optimizer/viaIR by default, and the large deploy script otherwise hits Solidity stack-depth limits.

Impact: Source mapping can be less precise than a normal optimized build, but the command consistently measures production contract coverage.

Recommendation: Use the same command locally, in CI, and in the checked-in coverage report.

Status: Acknowledged.

I-02: Fork Tests Need RPC Secrets

Severity: Informational.

Location: `test/fork/ForkIntegrations.t.sol`.

Description: Fork tests are present for USDC, Uniswap V2 router, and Chainlink feed integrations. They require `MAINNET_RPC_URL` to run against live network state.

Impact: Without the environment variable, local execution skips the live fork branch. CI can enable full fork execution by setting the secret.

Recommendation: Configure `MAINNET_RPC_URL` in GitHub Actions or run fork tests locally before final submission.

Status: Acknowledged.

G-01: Yul Math Helper

Severity: Gas.

Location: `src/libraries/RwaMath.sol`.

Description: AMM initial liquidity uses `sqrtYul` and keeps `sqrtSolidity` as a pure-Solidity equivalent for comparison.

Impact: Satisfies the inline Yul requirement while keeping the assembly block small and auditable.

Recommendation: Keep the benchmark in `script/GasCompare.s.sol` and document gas deltas in the gas report.

Status: Fixed / benchmarked.

6. Reproduced and Fixed Vulnerability Case Studies

Case Study A - Reentrancy

Item	Vulnerable version	Fixed version
Fixture	`test/mocks/ReentrancyFixtures.sol:VulnerableEthVault`	`test/mocks/ReentrancyFixtures.sol:FixedEthVault`
Vulnerability	Sends ETH with `call` before zeroing user balance.	Uses Checks-Effects-Interactions and `ReentrancyGuard`.
Impact	Attacker re-enters withdrawal and drains more ETH than deposited.	Attacker can only withdraw the owed amount once.
Proof tests	`test067_VulnerableReentrancyCaseStudyDrainsFunds`	`test068_FixedReentrancyCaseStudyKeepsAccounting`
Production relevance	Treasury and vault functions contain external value movement and require strict ordering.	Production flows zero state before calls or use non-reentrant modifiers.

Case Study B - Missing Access Control

Item	Vulnerable version	Fixed version
Fixture	`test/mocks/AccessControlFixtures.sol:VulnerableIssuer`	`test/mocks/AccessControlFixtures.sol:FixedIssuer`
Vulnerability	Anyone can mint asset tokens.	Minting requires `ISSUER_ROLE`.
Impact	Unbacked token supply can be created by arbitrary callers.	Only governance-approved issuers can mint.
Proof tests	`test069_VulnerableAccessControlCaseStudyAllowsAnyone`	`test070_FixedAccessControlCaseStudyRejectsAnyone`
Production relevance	RWA minting is the highest-risk privileged action.	`TBillToken.mint` is explicitly role-gated and issuer roles can be revoked by governance.

7. Centralization Analysis

The protocol intentionally centralizes long-term privileged execution through the Timelock rather than through an externally owned deployer account. This makes changes slower and visible, but governance is still a power center.

Power	Long-term holder	What can go wrong if malicious or compromised	Defense
Token upgrade	Timelock	Upgrade to malicious implementation.	Two-day delay, proposal visibility, storage-layout tests, isolated UUPS surface.
Issuer role grant/revoke	Timelock	Add a malicious issuer or remove legitimate issuers.	Proposal threshold, quorum, public vote, event trail.
Oracle feed update	Timelock	Set bad feed addresses or excessive staleness.	Timelock delay, adapter reverts invalid answers, frontend displays feed state.
Vault yield reporting	Timelock/manager	Report unsupported yield if manager lacks actual TBILL.	SafeERC20 transfer requires real token movement into the vault.
Treasury payments	Timelock	Schedule payment to attacker-controlled address.	Public proposal and queue lifecycle; pull-payment accounting is transparent.
Pause authority	Timelock	Halt transfers or vault flows.	Restricted scope, public governance action, documented emergency purpose.

If the deployer key is compromised before handoff, the attacker can grant roles, mint, pause, or change feed settings. After handoff, privileged actions require governance approval plus Timelock delay. The deployment verification script checks that no direct deployer admin backdoor remains.

8. Governance Attack Analysis

Flash-Loan Governance

ERC20Votes mitigates same-block voting power manipulation by using historical checkpoints. Voting power is measured at the proposal snapshot, not at the moment a user borrows tokens or executes a vote. This prevents a simple flash-loan borrow, vote, and repay sequence from creating durable voting power.

Whale Governance

Token governance remains plutocratic. A large holder can influence outcomes if they control enough delegated voting power. The protocol mitigates this with a 4% quorum, 1% proposal threshold, transparent proposal data, and a two-day execution delay. These controls do not eliminate whale risk, but they make hostile actions observable before execution.

Proposal Spam

Proposal creation requires at least 1% proposal threshold. This prevents dust holders from filling governance with spam proposals. Frontend proposal rendering still treats proposal data as untrusted and displays readable status rather than executing arbitrary data.

Timelock Bypass

The intended deployment state gives privileged roles to the Timelock, not to the deployer. `VerifyDeployment.s.sol` checks Timelock delay and role ownership. A bypass would require either a deployment mistake, a bug in the Timelock/Governor stack, or a privileged role left behind. The verification script is designed to catch that class before submission.

9. Oracle Attack Analysis

Threat	Risk	Defense	Residual risk
Stale price	Protocol reads an old answer and presents outdated reserve or price state.	`OracleAdapter` reverts when `updatedAt` is older than `maxStaleness`.	Governance can set `maxStaleness`; bad governance can weaken it.
Zero or negative answer	Bad feed returns invalid price/reserve value.	Adapter reverts on answer <= 0.	Feed malfunction can still cause denial of service by reverting.
Feed replacement attack	Governance points adapter to malicious feed.	Timelock delay and public proposal trail.	Token voting governance can still approve a bad feed if voters are malicious or careless.
AMM price manipulation	Users infer RWA price from pool spot price.	Protocol uses Chainlink-compatible feeds for oracle data, not AMM spot price.	Frontend must label AMM reserves separately from oracle price.
Testnet mock confusion	Mock reserve feed is mistaken for production proof of collateral.	Documentation and README explicitly state testnet feeds are mocks.	Production launch requires official feeds or institutional attestations.

10. CEI, Reentrancy, and External Call Review

Contract / function group	External call surface	Protection
`TBillVault.deposit/mint/withdraw/redeem`	ERC20 transfers	`nonReentrant`, OpenZeppelin ERC4626, SafeERC20.
`TBillVault.reportYield`	ERC20 transfer from manager	`nonReentrant`, role-gated, SafeERC20.
`RwaStableAMM.addLiquidity/removeLiquidity/swapExactIn`	ERC20 transfers	`nonReentrant`, SafeERC20, slippage checks, deadline checks, reserve updates.
`ProtocolTreasury.withdrawTokenPayment`	ERC20 transfer to payee	Owed amount zeroed before transfer; SafeERC20.
`ProtocolTreasury.withdrawEthPayment`	ETH call to payee	Owed amount zeroed before `call{value:}`, success checked.
`TBillToken.mint/burn/pause/unpause/setRegistry`	ERC20 state changes, no arbitrary external call	AccessControl roles, OpenZeppelin internals.
`OracleAdapter.latestPrice/latestReserve`	Static-like call to feed	Reverts stale, zero, negative, and incomplete data.
`ProtocolFactory.deployAssetCreate/Create2`	Contract creation	Role-gated deployment path and deterministic address prediction for CREATE2.

Forbidden primitive review:

- No `tx.origin` authorization.
- No `transfer` or `send` for ETH.
- No `block.timestamp` randomness.
- ERC20 protocol transfers use SafeERC20 where external tokens move.
- External calls handle return values or revert on failure.

11. Test and Coverage Evidence

Test class	Requirement	Evidence
Unit tests	At least 50	`test/Protocol.t.sol` contains broad positive and revert-path coverage across public functions.
Fuzz tests	At least 10	AMM swaps, vault deposit/withdraw rounding, and governance voting power are fuzzed.
Invariant tests	At least 5	AMM k, ERC20 supply conservation, vault accounting, treasury accounting, and role safety.
Fork tests	At least 3	USDC, Uniswap V2 router, and Chainlink feed fork hooks when `MAINNET_RPC_URL` is present.
Coverage	At least 90% line coverage	Current report: 92.31% line coverage for production `src/` contracts.
CI	All checks on push and pull request	GitHub Actions installs tooling and runs build, tests, coverage, Slither, and lint checks.

Coverage command:

```
forge coverage --ir-minimum --report summary --no-match-coverage 'script|test|src/mocks'
```

Current production-contract line coverage is documented in `docs/reports/coverage.md`.

12. Slither Appendix

CI command:

```
slither . --filter-paths "test|script|node_modules|lib" --exclude-dependencies
```

Submission requirement: zero High and zero Medium findings.

Current local note: the implementation machine did not have a preinstalled `slither` binary. The CI workflow installs `slither-analyzer` before running Slither. The final submission should paste the final CI Slither output below this appendix.

Severity	Expected final count	Handling requirement
Critical	0	Must be fixed before submission.
High	0	Must be fixed before submission.
Medium	0	Must be fixed before submission.
Low	Documented	Listed in findings table with explicit justification.
Informational	Documented	Listed in findings table or appendix with rationale.
Gas	Documented	Listed if meaningful and benchmarked where relevant.

13. Final Security Checklist

Control	Status
Privileged functions guarded	Complete - AccessControl/Governor/Timelock used.
CEI or ReentrancyGuard	Complete - value-moving flows reviewed and tested.
SafeERC20	Complete - external ERC20 transfers use SafeERC20.
Oracle staleness	Complete - stale, zero, negative, and incomplete feed data reverts.
UUPS storage collision proof	Complete - V2 appends storage and upgrade test preserves V1 state.
Reentrancy case study	Complete - vulnerable and fixed tests included.
Access-control case study	Complete - vulnerable and fixed tests included.
No forbidden primitives	Complete - no `tx.origin`, no ETH `transfer/send`, no timestamp randomness.
Timelock admin handoff	Complete for Anvil; Base Sepolia verification requires deploy secrets.
Slither High/Medium	CI configured for zero High/Medium; final CI output to be appended.